



CourseCube®
(Formerly Java Learning Center)

Spring Boot 2

Module 1

Data Access with Boot - Part 1

Author

Srinivas Dande





Module 1 : Spring Data Access with Boot - Part 1

Objectives: In this Module, You will learn Data Access with Spring Boot Applications with JDBC, Data Sources and Various Spring Boot features.

Topics:

- ✓ Spring -JDBC Flow
- ✓ Developing Spring -JDBC Example
- ✓ Understanding the Problems with this Example.
- ✓ First Spring Boot Example
- ✓ Spring Boot Features.
- ✓ Starters
- ✓ Auto Configurations
- ✓ Exploring Starters
- ✓ Exploring Auto Configurations
- ✓ Exploring YAML Files
- ✓ Using Hikari DataSource
- ✓ Using Tomcat DataSource
- ✓ Using DBCP2
- ✓ Using DBCP
- ✓ Using JdbcTemplate
- ✓ Using NamedParameterJdbcTemplate
- ✓ Using Logging with Boot
- ✓ Understanding Spring Boot Start-Up Tasks
- ✓ Exploring CommandLineRunner
- ✓ Exploring @SpringBootApplication
- ✓ Exploring @ComponentScan



Spring Boot 2

- 1) Spring offers a powerful way to build web applications, with support for dependency injection, transaction management, polyglot persistence, application security, first-hand REST API support, an MVC framework and a lot more.
- 2) Spring applications have always required significant configuration and, which sometimes build up a lot of complexity during development. That's where Spring Boot comes in.
- 3) The Spring Boot project aims to make building web application with Spring much faster and easier. The guiding principle of Boot is convention over configuration.

Important features in Spring Boot:

1. **Spring Boot Starters**
2. **Auto Configurations**
3. **Spring Boot Initializer**
4. **Spring Boot CLI**
5. **Dev Tools**
6. **Externalized Configurations**
7. **YAML Support**
8. **Embedded Containers**
9. **Actuators**
10. **Spring Boot Admin**



demo1-spring-jdbc: Spring and JDBC with Maven

demo1-spring-jdbc: Working Steps:

A) Setup Database

```
create database myjlcdb5;
use myjlcdb5;
create table mycustomers(
cid int primary key,
cname char(10),
email char(10),
phone long,
city char(10)
);
```

B) Create the maven Project.

- 1) Open the Eclipse required Workspace
- 2) Select New -> Maven Project.
- 3) Check the checkbox for create Simple Project (skip the Archetype selection) and Click on Next Button.
- 4) Provide the following
Group Id: **com.jlcindia.spring**
Artifact Id : **demo1-spring-jdbc**
Version: **1.0**
packaging: **jar**
Name: **demo1-spring-jdbc**
and Click On finish button.

- 5) Add the maven-compiler-plugin

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.1</version>
  <configuration>
    <source>1.8</source>
    <target>1.8</target>
  </configuration>
</plugin>
```

- 6) You can see the Error in Maven Project.

Fix the Error by updating the Maven Project.

Select the Project, right click and then select Maven -> Update Project.



7) Update pom.xml by adding the following dependencies.

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>5.2.5.RELEASE</version>
</dependency>

<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.2.5.RELEASE</version>
</dependency>

<dependency>
    <groupId>commons-dbcp</groupId>
    <artifactId>commons-dbcp</artifactId>
    <version>1.4</version>
</dependency>

<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.6</version>
</dependency>
```

8) Create the Package called **com.jlcindia.spring**

9) Write JLCAppConfig.java (Bean Configuration class)

- Configure DataSource.
- Configure JdbcTemplate

10)Write the following Files

- Customer.java
- CustomerRowMapper.java

11)Write the DAO and its Implementation class

- CustomerDAO.java
- CustomerDAOImpl.java

12)Write Client Program - MyClient and Run it.



Spring -JDBC Example with Maven

demo1-spring-jdbc: Files required

1. MyClient.java	
2. CustomerDAO.java	
3. CustomerDAOImpl.java	
4. Customer.java	
5. CustomerRowMapper.java	
6. JLCAppConfig.java	
7. pom.xml	

1)MyClient.java

```
package com.jlcindia.spring;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class MyClient {
    public static void main(String[] args) {

        ApplicationContext ctx=new AnnotationConfigApplicationContext(JLCAppConfig.class);
        CustomerDAO custDAO =(CustomerDAO) ctx.getBean("custDAO");

        Customer cust = new Customer(104,"hello","hello@jlc",12345,"Delhi");
        custDAO.addCustomer(cust);

        List<Customer> mylist1 = custDAO.getAllCustomers();
        mylist1.forEach(mycust -> System.out.println(mycust));
        System.out.println("-----");

        List<Customer> mylist2 = custDAO.getCustomersByCity("Blore");
        mylist2.forEach(mycust -> System.out.println(mycust));

        System.out.println("Done!!!");
    }
}
```



2) CustomerDAO.java

```
package com.jlcindia.spring;

import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO {
    public void addCustomer(Customer cust);
    public List<Customer> getAllCustomers();
    public List<Customer> getCustomersByCity(String city);
}
```

3) CustomerDAOImpl.java

```
package com.jlcindia.spring;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository("custDAO")
public class CustomerDAOImpl implements CustomerDAO{

    @Autowired
    JdbcTemplate jtemp;

    @Override
    public void addCustomer(Customer cust) {
        String SQL="insert into mycustomers values(?,?,?,?)";
        jtemp.update(SQL,
            cust.getCid(),cust.getCname(),cust.getEmail(),cust.getPhone(),cust.getCity());
    }

    @Override
    public List<Customer> getAllCustomers() {
        String SQL="select * from mycustomers";
        List<Customer> mylist = jtemp.query(SQL, new CustomerRowMapper());
        return mylist;
    }
}
```



```
@Override
public List<Customer> getCustomersByCity(String city) {
    String SQL="select * from mycustomers where city=?";
    List<Customer> mylist = jtemp.query(SQL, new CustomerRowMapper(),city);
    return mylist;
}

}
```

4)Customer.java

```
package com.jlcindia.spring;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Customer {

    private int cid;
    private String cname;
    private String email;
    private int phone;
    private String city;

    public Customer() {}
    public Customer(String cname, String email, int phone, String city) {
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
    }

    public Customer(int cid, String cname, String email, int phone, String city) {
        super();
        this.cid = cid;
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
    }

    //Setters and Getters
    @Override
    public String toString() {
        return "[" + cid + ", " + cname + ", " + email + ", " + phone + ", " + city + "]";
    }
}
```



5)CustomerRowMapper.java

```
package com.jlcindia.spring;

import java.sql.ResultSet;
import java.sql.SQLException;

import org.springframework.jdbc.core.RowMapper;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class CustomerRowMapper implements RowMapper<Customer> {

    @Override
    public Customer mapRow(ResultSet rs, int rowNum) throws SQLException {
        Customer cust=new Customer();

        cust.setCid(rs.getInt(1));
        cust.setCname(rs.getString(2));
        cust.setEmail(rs.getString(3));
        cust.setPhone(rs.getInt(4));
        cust.setCity(rs.getString(5));

        return cust;
    }
}
```

6)JLCAppConfig.java

```
package com.jlcindia.spring;

import javax.sql.DataSource;

import org.apache.commons.dbcp.BasicDataSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Configuration
@ComponentScan(basePackages = {"com.jlcindia.spring"})
public class JLCAppConfig {
```



```
@Bean(name = "dataSource")
```

```
public DataSource getDataSource() {
```

```
    BasicDataSource bds = new BasicDataSource();  
    bds.setDriverClassName("com.mysql.jdbc.Driver");  
    bds.setUrl("jdbc:mysql://localhost:3306/myjlcdb5");  
    bds.setUsername("root");  
    bds.setPassword("srinivas");  
    bds.setMaxActive(5);  
    return bds;  
}
```

```
@Bean(name = "jtemp")
```

```
public JdbcTemplate getJdbcTemplate(DataSource dataSource) {  
    JdbcTemplate jtemp = new JdbcTemplate(dataSource);  
    return jtemp;  
}  
}
```

7)pom.xml

```
<project ...>
```

```
<modelVersion>4.0.0</modelVersion>  
<groupId>com.jlcindia.spring</groupId>  
<artifactId>demo1-spring-jdbc</artifactId>  
<version>1.0</version>
```

```
<name>demo1-spring-jdbc</name>  
<description>This is Spring and JDBC example</description>  
<url>https://www.jlcindia.com</url>
```

```
<properties>  
<java.version>1.8</java.version>  
<spring.version>5.2.5.RELEASE</spring.version>  
</properties>
```

```
<dependencies>
```

```
<dependency>  
<groupId>org.springframework</groupId>  
<artifactId>spring-context</artifactId>  
<version>${spring.version}</version>  
</dependency>
```



```
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-jdbc</artifactId>
<version>${spring.version}</version>
</dependency>

<dependency>
<groupId>commons-dbcp</groupId>
<artifactId>commons-dbcp</artifactId>
<version>1.4</version>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.1</version>
<configuration>
<source>${java.version}</source>
<target>${java.version}</target>
</configuration>
</plugin>
</plugins>
</build>
</project>
```



Problems:

- 1) We need to specify many dependencies for getting different Jars belongs to different technologies. (with pom.xml)
=> What the Jars to download
- 2) We need to check Version Compatibility and need to specify the Version Compatibility (with pom.xml)
=> What version of Jars to download
- 3) Complexity in Configuring many Beans in Bean Configuration class
- 4) Hard-Coding Bean Properties in Bean Configuration class

How SpringBoot Solves the Above Problems.

1) Starters.

- Gets All the Dependent Compatible Jars with given starter name.
 - ✓ spring-boot-starter-jdbc
 - ✓ spring-boot-starter-data-jpa
 - ✓ spring-boot-starter-data-mongodb
 - ✓ spring-boot-starter-web
 - ✓ spring-boot-starter-thymeleaf
 - ✓ spring-boot-starter-security
 - ✓ spring-boot-starter-mail
 - ✓ spring-boot-starter-test
- and Many more

2) AutoConfiguration

- Most of the Beans will be configured by the Spring Container automatically.
- Beans will be configured conditionally based availability of Classes in the Classpath.

3) Centralizing the Configuration Data

- You can centralize the Configuration Data/Parameters in Property Files or YAML Files



demo2-boot-jdbc: Spring Boot and JDBC

demo2-boot-jdbc: Working Steps:

A) Setup Database

```
create database myjdbc5;  
use myjdbc5;  
create table mycustomers(  
  cid int primary key,  
  cname char(10),  
  email char(10),  
  phone long,  
  city char(10)  
);
```

B) Create the maven Project.

- 1) Open the Eclipse required Workspace
- 2) Select New -> Maven Project.
- 3) Check the checkbox for create Simple Project (skip the Archetype selection) and Click on Next Button.
- 4) Provide the following
Group Id: **com.jlcindia.springboot**
Artifact Id : **demo2-boot-jdbc**
Version : 1.0
packaging : jar
Name : **demo2-boot-jdbc**

and Click On finish button.

- 5) Remove the following two Source Folders.

- a. src/test/java
- b. src/test/resources

C) Start the Example Code.

- 1) Add the spring-boot-maven-plugin in pom.xml

```
<plugin>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-maven-plugin</artifactId>  
</plugin>
```



2) Update pom.xml by adding the following dependencies.

```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
</dependency>

</dependencies>
```

3) You can see the Error in Maven Project.

Fix the Error by updating the Maven Project.

Select the Project, right click and then select Maven -> Update Project.

4) Create **application.properties** under **src/main/resources**

5) Add the following properties in application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

#Hikari Properties

```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5
```

#Log Properties

```
logging.level.root=DEBUG
logging.pattern.console=%-5level %logger{36} - %msg%n
```

6) Create the Package called **com.jlcindia.springboot**

7) Write JLCAppConfig.java (Bean Configuration class)

- No Beans to Configure

@SpringBootApplication

```
public class JLCAppConfig {
```

```
}
```

8) Write the following Files (**Copy from Demo1**)

- Customer.java
- CustomerRowMapper.java

9) Write the DAO and its Implementation class (**Copy from Demo1**)

- CustomerDAO.java
- CustomerDAOImpl.java

10) Write Boot Main Class - **MyBootApplication** and Run it.

Run As => Spring Boot Application

Spring Boot-JDBC Example with Maven

demo2-boot-jdbc: Files required

1. MyBootApplication.java	Newly Added in Demo2
2. CustomerDAO.java	Same As Demo1
3. CustomerDAOImpl.java	Same As Demo1
4. Customer.java	Same As Demo1
5. CustomerRowMapper.java	Same As Demo1
6. JLCAppConfig.java	Updated in Demo2
7. Application.properties	Newly Added in Demo2
8. pom.xml	Updated in Demo2



1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        // 1.Add the Customer
        Customer cust = new Customer(106, "hai", "hai@jlc", 12345, "Blore");
        custDAO.addCustomer(cust);

        // 2.get All Customers
        List<Customer> mylist1 = custDAO.getAllCustomers();
        mylist1.forEach(mycust -> System.out.println(mycust));
        System.out.println("-----");

        // 3.Get Customers By City
        List<Customer> mylist2 = custDAO.getCustomersByCity("Blore");
        mylist2.forEach(mycust -> System.out.println(mycust));

        System.out.println("Done!!!");
    }

    public static void main(String[] args) {
        SpringApplication.run(MyBootApplication.class, args);
    }
}
```



6)JLAppConfig.java

```
package com.jlcindia.springboot;

import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@SpringBootApplication
public class JLAppConfig {
}
```

7)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

#Hikari Properties

```
spring.datasource.hikari.maximum-pool-size=100
spring.datasource.hikari.idle-timeout=20000
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n
```

8)pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo2-boot-jdbc</artifactId>
<version>1.0</version>
<name>demo2-boot-jdbc</name>
<description>This is My First Spring Boot Application</description>
<url>https://www.jlcindia.com</url>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Using Tomcat DataSource

demo3-boot-jdbc : Working Steps:

- 1) Copy demo2-boot-jdbc and Paste as demo3-boot-jdbc
- 2) Update the pom.xml

A) Exclude HikariCP

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
<exclusions>
<exclusion>
<groupId>com.zaxxer</groupId>
<artifactId>HikariCP</artifactId>
</exclusion>
</exclusions>
</dependency>
```

B) Add Dependency for Tomcat CP.

```
<dependency>
<groupId>org.apache.tomcat</groupId>
<artifactId>tomcat-jdbc</artifactId>
</dependency>
```

- 3) Update **application.properties**

A) Remove Hikari Properties

B) Add Tomcat Properties as follows

```
spring.datasource.tomcat.initialSize=5
spring.datasource.tomcat.max-active=25
```

- 4) Run the Application.

Run As => Spring Boot Application

demo3-boot-jdbc: Using Tomcat DataSource

demo3-boot-jdbc: Files required

1. MyBootApplication.java	Same As Demo2
2. CustomerDAO.java	Same As Demo2
3. CustomerDAOImpl.java	Same As Demo2
4. Customer.java	Same As Demo2
5. CustomerRowMapper.java	Same As Demo2
6. application.properties	Updated in Demo3
7. pom.xml	Updated in Demo3



6)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb
spring.datasource.username=root
spring.datasource.password=srinivas
```

#Tomcat Properties

```
spring.datasource.tomcat.initialSize=5
spring.datasource.tomcat.max-active=25
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

7)pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo3-boot-jdbc</artifactId>
<version>1.0</version>

<name>demo3-boot-jdbc</name>
<description>My Spring Boot Application</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
<exclusions>
<exclusion>
<groupId>com.zaxxer</groupId>
```



```
<artifactId>HikariCP</artifactId>
</exclusion>
</exclusions>
</dependency>

<dependency>
<groupId>org.apache.tomcat</groupId>
<artifactId>tomcat-jdbc</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Using DBCP2 DataSource

demo4-boot-jdbc: Working Steps:

1) Copy demo3-boot-jdbc and Paste as demo4-boot-jdbc

2) Update the pom.xml

A) Exclude HikariCP

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
  <exclusions>
    <exclusion>
      <groupId>com.zaxxer</groupId>
      <artifactId>HikariCP</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

B) Add Dependency for DBCP2

```
<dependency>
<groupId>org.apache.commons</groupId>
<artifactId>commons-dbcp2</artifactId>
</dependency>
```

3) Update **application.properties**

A) Remove Tomcat Properties

B) Add DBCP2 Properties as follows

```
spring.datasource.dbcp2.initial-size=5
spring.datasource.dbcp2.max-total=20
spring.datasource.dbcp2.pool-prepared-statements=true
```

4) Run the Application.

Run As => Spring Boot Application



demo4-boot-jdbc: Using DBCP2 DataSource

demo4-boot-jdbc: Files required

1. MyBootApplication.java	Same As Demo3
2. CustomerDAO.java	Same As Demo3
3. CustomerDAOImpl.java	Same As Demo3
4. Customer.java	Same As Demo3
5. CustomerRowMapper.java	Same As Demo3
6. application.properties	Updated in Demo4
7. pom.xml	Updated in Demo4

6)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb
spring.datasource.username=root
spring.datasource.password=srinivas
```

#DBCP2 Properties

```
spring.datasource.dbcp2.initial-size=5
spring.datasource.dbcp2.max-total=20
spring.datasource.dbcp2.pool-prepared-statements=true
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

7)pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo4-boot-jdbc</artifactId>
<version>1.0</version>
<name>demo4-boot-jdbc</name>
<description>My Spring Boot Application</description>
<url>https://www.jlcindia.com</url>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>
```



```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
<exclusions>
<exclusion>
<groupId>com.zaxxer</groupId>
<artifactId>HikariCP</artifactId>
</exclusion>
</exclusions>
</dependency>

<dependency>
<groupId>org.apache.commons</groupId>
<artifactId>commons-dbcp2</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Using Hikari, Tomcat and DBCP2 DataSources

demo5-boot-jdbc : Working Steps:

1) Copy **demo4-boot-jdbc** and Paste as **demo5-boot-jdbc**

2) Update the pom.xml

A) Include HikariCP

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

B) Add Dependency for DBCP2

```
<dependency>
<groupId>org.apache.commons</groupId>
<artifactId>commons-dbcp2</artifactId>
</dependency>
```

C) Add Dependency for Tomcat CP.

```
<dependency>
<groupId>org.apache.tomcat</groupId>
<artifactId>tomcat-jdbc</artifactId>
</dependency>
```

3) Update **application.properties**

A) Add Hikari Properties as follows

```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5
```

B) Add Tomcat Properties as follows

```
spring.datasource.tomcat.initialSize=5
spring.datasource.tomcat.max-active=25
```

C) Add DBCP2 Properties as follows

```
spring.datasource.dbcp2.initial-size=5
spring.datasource.dbcp2.max-total=20
spring.datasource.dbcp2.pool-prepared-statements=true
```

4) Run the Application.

Run As => Spring Boot Application



demo5-boot-jdbc: Using 3 DataSource

demo5-boot-jdbc-: Files required

1. MyBootApplication.java	Same As Demo4
2. CustomerDAO.java	Same As Demo4
3. CustomerDAOImpl.java	Same As Demo4
4. Customer.java	Same As Demo4
5. CustomerRowMapper.java	Same As Demo4
6. application.properties	Updated in Demo5
7. pom.xml	Updated in Demo5

6)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb
spring.datasource.username=root
spring.datasource.password=srinivas
```

#Hikari Properties

```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5
```

#Tomcat Properties

```
spring.datasource.tomcat.initialSize=5
spring.datasource.tomcat.max-active=25
```

#DBCP2 Properties

```
spring.datasource.dbcp2.initial-size=5
spring.datasource.dbcp2.max-total=20
spring.datasource.dbcp2.pool-prepared-statements=true
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

7)pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo5-boot-jdbc</artifactId>
<version>1.0</version>
```



```
<name> demo5-boot-jdbc </name>
<description> My Spring Boot Application</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
<groupId>org.apache.tomcat</groupId>
<artifactId>tomcat-jdbc</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Using Tomcat and DBCP2 DataSources

demo6-boot-jdbc: Working Steps:

- 1) Copy **demo5-boot-jdbc** and Paste as **demo6-boot-jdbc**
- 2) Update the pom.xml

A) Include HikariCP

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
<exclusions>
<exclusion>
<groupId>com.zaxxer</groupId>
<artifactId>HikariCP</artifactId>
</exclusion>
</exclusions>
</dependency>
```

B) Add Dependency for DBCP2

```
<dependency>
<groupId>org.apache.commons</groupId>
<artifactId>commons-dbcp2</artifactId>
</dependency>
```

C) Add Dependency for Tomcat CP.

```
<dependency>
<groupId>org.apache.tomcat</groupId>
<artifactId>tomcat-jdbc</artifactId>
</dependency>
```

- 3) Update **application.properties**

A) Add Tomcat Properties as follows

```
spring.datasource.tomcat.initialSize=5
spring.datasource.tomcat.max-active=25
```

B) Add DBCP2 Properties as follows

```
spring.datasource.dbcp2.initial-size=5
spring.datasource.dbcp2.max-total=20
spring.datasource.dbcp2.pool-prepared-statements=true
```

- 4) Run the Application.

Run As => Spring Boot Application



demo6-boot-jdbc: Using 2 DataSource

demo6-boot-jdbc: Files required

1. MyBootApplication.java	Same As Demo5
2. CustomerDAO.java	Same As Demo5
3. CustomerDAOImpl.java	Same As Demo5
4. Customer.java	Same As Demo5
5. CustomerRowMapper.java	Same As Demo5
6. application.properties	Updated in Demo6
7. pom.xml	Updated in Demo6

6)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb
spring.datasource.username=root
spring.datasource.password=srinivas
```

#Tomcat Properties

```
spring.datasource.tomcat.initialSize=5
spring.datasource.tomcat.max-active=25
```

#DBCP2 Properties

```
spring.datasource.dbcp2.initial-size=5
spring.datasource.dbcp2.max-total=20
spring.datasource.dbcp2.pool-prepared-statements=true
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

7)pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo6-boot-jdbc</artifactId>
<version>1.0</version>
<name>demo6-boot-jdbc</name>
<description> My Spring Boot Application</description>
<url>https://www.jlcindia.com</url>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
<groupId>org.apache.tomcat</groupId>
<artifactId>tomcat-jdbc</artifactId>
</dependency>

<dependency>
<groupId>org.apache.commons</groupId>
<artifactId>commons-dbcp2</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Using NamedParameterJdbcTemplate

demo7-boot-jdbc: Files required

1. MyBootApplication.java	Updated in Demo7
2. CustomerDAO.java	Updated in Demo7
3. CustomerDAOImpl.java	Updated in Demo7
4. Customer.java	Same As Demo6
5. CustomerRowMapper.java	Same As Demo6
6. application.properties	Updated in Demo7
7. pom.xml	Updated in Demo7

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        // 1.Add the Customer
        Customer cust = new Customer(107, "aaa", "aaa@jlc", 111, "Delhi");
        custDAO.addCustomer(cust);

        // 2.get All Customers
        List<Customer> mylist1 = custDAO.getAllCustomers();
        mylist1.forEach(mycust -> System.out.println(mycust));
        System.out.println("-----");
    }
}
```



// 3.Get Customers By City

```
List<Customer> mylist2 = custDAO.getCustomersByCity("Delhi");  
mylist2.forEach(mycust -> System.out.println(mycust));  
System.out.println("-----");
```

// 4.Get Customers Email By Phone

```
String email = custDAO.getCustomerEmailByPhone(111);  
System.out.println(email);  
System.out.println("Done!!!");  
}
```

```
public static void main(String[] args) {  
    SpringApplication.run(MyBootApplication.class, args);  
}  
}
```

2)CustomerDAO.java

```
package com.jlcindia.springboot;  
  
import java.util.List;  
import java.util.Map;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
public interface CustomerDAO {  
    public void addCustomer(Customer cust);  
    public void addCustomer(Map<String,Object> map);  
    public List<Customer> getAllCustomers();  
    public List<Customer> getCustomersByCity(String city);  
    public String getCustomerEmailByPhone(int phone);  
}
```

3) CustomerDAOImpl.java

```
package com.jlcindia.springboot;  
  
import java.util.*;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.jdbc.core.JdbcTemplate;  
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;  
import org.springframework.stereotype.Repository;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */
```



@Repository

```
public class CustomerDAOImpl implements CustomerDAO{
```

@Autowired

```
NamedParameterJdbcTemplate namedJTemp;
```

@Autowired

```
JdbcTemplate jtemp;
```

@Override

```
public void addCustomer(Customer cust) {
```

```
String SQL="insert into mycustomers values(:mycid,:mycname,:myemail,:myphone,:mycity)";
```

```
Map<String,Object> params = new HashMap<>();
```

```
params.put("mycid", cust.getCid());
```

```
params.put("mycname", cust.getCname());
```

```
params.put("myemail", cust.getEmail());
```

```
params.put("myphone", cust.getPhone());
```

```
params.put("mycity", cust.getCity());
```

```
namedJTemp.update(SQL, params);
```

```
}
```

@Override

```
public void addCustomer(Map<String, Object> map) {
```

```
String SQL="insert into mycustomers values(:mycid,:mycname,:myemail,:myphone,:mycity)";
```

```
namedJTemp.update(SQL, map);
```

```
}
```

@Override

```
public List<Customer> getAllCustomers() {
```

```
String SQL="select * from mycustomers";
```

```
List<Customer> mylist = namedJTemp.query(SQL, new CustomerRowMapper());
```

```
return mylist;
```

```
}
```

@Override

```
public List<Customer> getCustomersByCity(String city) {
```

```
String SQL="select * from mycustomers where city=:mycity";
```

```
Map<String,Object> params = new HashMap<>();
```

```
params.put("mycity", city);
```

```
List<Customer> mylist = namedJTemp.query(SQL, params,new CustomerRowMapper());
```

```
return mylist;
```

```
}
```



```
public String getCustomerEmailByPhone(int phone) {  
    String SQL="select email from mycustomers where phone=:myphone";  
  
    Map<String,Object> params = new HashMap<>();  
    params.put("myphone", phone);  
  
    String email = namedJTemp.queryForObject(SQL, params, String.class);  
    return email;  
}  
  
}
```

6)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5  
spring.datasource.username=root  
spring.datasource.password=srinivas
```

#Hikari Properties

```
spring.datasource.hikari.maximum-pool-size=100  
spring.datasource.hikari.idle-timeout=20000
```

#Log Properties

```
logging.level.root=DEBUG  
logging.pattern.console=%-5level %logger{36} - %m %n
```

7)pom.xml

```
<project ...>  
  
    <modelVersion>4.0.0</modelVersion>  
    <groupId>com.jlcindia.springboot</groupId>  
    <artifactId>demo7-boot-jdbc</artifactId>  
    <version>1.0</version>  
  
    <name>demo7-boot-jdbc</name>  
    <description>This is My Spring Boot Application</description>  
    <url>https://www.jlcindia.com</url>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Understanding Spring Boot Start-Up Tasks

demo8-boot-startup-tasks: Working Steps

- 1) Copy demo7-boot-jdbc as demo8-boot-startup-tasks
- 2) Remove all except the following
 - **MyBootApplication.java** - Boot Main Class + Bean Configuration class
 - **JLCAppConfig.java** - Bean Configuration class
- 3) Write following DAO's in com.jlcindia.dao
 - **HelloDAO.java**
 - **HaiDAO.java**
- 4) Write following Services in com.jlcindia.service
 - **HelloService.java**
 - **HaiService.java**
- 5) Write the following DTO in com.jlcindia.to
 - **Hello.java**
 - **Hai.java**
- 6) Inject the Following Beans in MyBootApplication
 - **HaiDAO**
 - **HaiService**
 - **HelloDAO**
 - **HelloService**
 - **Hello**
 - **Hai**

 - **DataSource**
 - **JdbcTemplate**
 - **DataSourceTransactionManager**
 - **NamedParameterJdbcTemplate**
 - **ApplicationContext**
- 7) Check whether following Bean Instances are running in Spring Container or Not?
 - **MyBootApplication**
 - **JLCAppConfig**
- 8) List All the Names of the Beans running in Spring Container



demo8-boot-startup-tasks: Files required

1. MyBootApplication.java	2. JLCAppConfig.java
3. HelloDAO.java	4. HaiDAO.java
5. HelloService.java	6. HaiService.java
7. Hello.java	8. Hai.java
9. application.properties	10. pom.xml

1) MyBootApplication.java

```
package com.jlcindia;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.Bean;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;

import com.jlcindia.dao.HaiDAO;
import com.jlcindia.dao.HelloDAO;
import com.jlcindia.service.HaiService;
import com.jlcindia.service.HelloService;
import com.jlcindia.to.Hai;
import com.jlcindia.to.Hello;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    ApplicationContext ctx;

    @Autowired
    DataSource dataSource;

    @Autowired
    JdbcTemplate jtemp;
```



```
@Autowired
NamedParameterJdbcTemplate npjtemp;

@Autowired
DataSourceTransactionManager txManager;

@Autowired(required = false)
HelloDAO helloDAO;

@Autowired(required = false)
HelloService helloService;

@Autowired
HaiDAO haiDAO;

@Autowired
HaiService haiService;

@Autowired
Hello hello;

@Autowired
Hai hai;

@Override
public void run(String... args) throws Exception {
    System.out.println("1." + ctx);
    System.out.println("2. Bean Count : " + ctx.getBeanDefinitionCount());
    System.out.println("3." + dataSource);
    System.out.println("4." + jtemp);
    System.out.println("5." + npjtemp);
    System.out.println("6." + txManager);
    System.out.println("7." + helloDAO);
    System.out.println("8." + helloService);
    System.out.println("9." + haiDAO);
    System.out.println("10." + haiService);
    System.out.println("11." + hello);
    System.out.println("12." + hai);
    System.out.println("-----");
    System.out.println("13. Bean Count : " + ctx.getBeanDefinitionCount());
    String beanNames[] = ctx.getBeanDefinitionNames();

    for(String beanName: beanNames) {
        System.out.println(beanName);
    }
    System.out.println("Done!!!");
}
```



```
public static void main(String[] args) {
    System.out.println("Main Begin");
    SpringApplication.run(MyBootApplication.class, args);
    System.out.println("Main end");
}

@Bean(name="myhello1")
public Hello getHello1() {
    return new Hello();
}

@Bean(name="myhai1")
public Hai getHai1() {
    return new Hai();
}

@Bean(name="myhello2")
public Hello getHello2() {
    return new Hello();
}

@Bean(name="myhai2")
public Hai getHai2() {
    return new Hai();
}
}
```

2)JLAppConfig.java

```
package com.jlcindia;

import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

import com.jlcindia.dao.HaiDAO;
import com.jlcindia.service.HaiService;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class JLAppConfig {

    @Bean(name="myhaiDAO")
    public HaiDAO getHaiDAO() {
        return new HaiDAO();
    }
}
```



```
@Bean(name="myhaiService")
public HaiService getHaiService() {
    return new HaiService();
}
}
```

3) HelloDAO.java

```
package com.jlcindia.dao;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@Repository
public class HelloDAO {

    @Autowired(required = false)
    JdbcTemplate jtemp;

    @Autowired(required = false)
    DataSource ds;

    public HelloDAO() {
        System.out.println("HelloDAO-D.C");
        System.out.println("A." + ds);
        System.out.println("B." + jtemp);
    }
}
```

4) HaiDAO.java

```
package com.jlcindia.dao;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center

```



```
*/  
  
public class HaiDAO {  
  
    @Autowired(required = false)  
    JdbcTemplate jtemp;  
  
    @Autowired(required = false)  
    DataSource ds;  
  
    public HaiDAO() {  
        System.out.println("HaiDAO-D.C");  
        System.out.println("A." + ds);  
        System.out.println("B." + jtemp);  
    }  
  
}
```

5) HelloService.java

```
package com.jlcindia.service;  
  
import javax.sql.DataSource;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.jdbc.core.JdbcTemplate;  
import org.springframework.stereotype.Service;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
@Service  
public class HelloService {  
  
    @Autowired(required = false)  
    JdbcTemplate jtemp;  
  
    @Autowired(required = false)  
    DataSource ds;  
  
    public HelloService(){  
        System.out.println("HelloService-D.C");  
        System.out.println("A." + ds);  
        System.out.println("B." + jtemp);  
    }  
  
}
```



6) HaiService.java

```
package com.jlcindia.service;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class HaiService {

    @Autowired(required = false)
    JdbcTemplate jtemp;

    @Autowired(required = false)
    DataSource ds;

    public HaiService(){
        System.out.println("HaiService-D.C");
        System.out.println("A." + ds);
        System.out.println("B." + jtemp);
    }
}
```

7) Hello.java

```
package com.jlcindia.to;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Component
public class Hello {

    @Autowired(required = false)
    JdbcTemplate jtemp;
```



```
@Autowired(required = false)
DataSource ds;

public Hello() {
    System.out.println("Hello-D.C");
    System.out.println("A." + ds);
    System.out.println("B." + jtemp);
}
}
```

8) Hai.java

```
package com.jlcindia.to;

import javax.sql.DataSource;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Component
public class Hai {

    @Autowired(required = false)
    JdbcTemplate jtemp;

    @Autowired(required = false)
    DataSource ds;

    public Hai() {
        System.out.println("Hai-D.C");
        System.out.println("A." + ds);
        System.out.println("B." + jtemp);
    }
}
```

9)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```



#Log Properties

logging.level.root=INFO

logging.pattern.console=%-5level %logger{36} - %m %n

10)pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia </groupId>
<artifactId>demo8-boot-startup-tasks</artifactId>
<version>1.0</version>
<name>demo8-boot-startup-tasks</name>
<description>This is My Spring Boot Application</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>
</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Interview Questions

Q1) What is Spring Boot?

Ans:

Spring Boot is a Framework to Develop Web Applications using Spring Framework.

Q2) What are the Spring Boot features discussed till now?

Ans:

- 1) Spring Boot Starters
- 2) Auto Configuration
- 3) Centralizing Configuration Data with Property File /YAML

Q3) What is use of Spring Boot Starters?

Ans:

Each Starter brings some set of Jars and it's Dependent and Compatible jars.

Q4) What is Auto Configuration?

Ans:

Beans will be configured by the Spring Boot automatically without explicit Configuration. So Bean Instances will be available in Spring Container Readily. You have to inject those beans where ever you want.

Q5) Can I ask the Spring Boot to Auto-Configure CustomerDAOImpl?

Ans:

No

Q6) Is it required to me to Configure CustomerDAOImpl in Bean Configuration class?

Ans:

No

Q7) How Spring Boot will decide which bean to Auto-Configure?

Ans:

List of Beans will be Auto-Configured based on the Availability of Bean class in the Class Path.

- Already Some List of Beans are Prepared by Boot Team. (consider 100 Beans)
- At Boot Application Start-Up, Boot Scans All the Packages in the Classpath to Check whether these Bean Classes are available in class path or not.
- Prepares CONDITIONS EVALUATION REPORT with Positive Matches and Negative Matches.
- You can see the CONDITIONS EVALUATION REPORT in debug mode.



Q8) How to Configure the Beans which I am writing In My applications?

Ans:

A) Some Beans will be Instanciated with ComponentScan facility.

Your Application specific beans like Controllers,Business Services,DAO etc can be marked with Sterotype annotations called @Controller,@Service, @Repository .

B) Some Beans will be Auto-Configured by the Spring Boot.

All the Built-In Beans like JdbcTemplate, NamedParameterJdbcTemplate, DataSourceTransactionManager, EntityManagerFactory, EntityManager, JpaTransactionManager, JpaTemplate, DispatcherServlet, RabbitMQTemplate, KafkaTemplate

C) Some Beans has to be configured by You explicitly.

If above two cases are not possible then you have to configure the Bean explicitly.

Q9) What is the use of Property File / YAML File?

Ans :

To Centralize the Bean Configuration Data.

Q10) Can I have both files (properties and yaml)?

Ans:

Any one is enough, but you can write both.

Q11) How Can I Spring Application in Oracle?

Ans:

Using Maven Dependency , You can not get the Oracle Jar from Maven Global Repo.

- Download Oracle Jar file.
- Push Your jar into Maven Local repo by Specifying Group Id, Artifact Id,Version
- Specify the Maven Dependency in pom.xml with Above Specified Group Id, Artifact Id,Version
- You can See Oracle Jar in your Project Dependencies.

Q12) What are various Third Party DataSources you know?

Ans: Following are various **Connection Pooling Algorithms/Thind Party DataSources**

- Hikari DataSource
- Tomcat DataSource
- DBCP2
- DBCP
- C3P0 DataSources



Q13) Can I configure two Databases related Properties in One Boot Application?

Ans:

No, You can write Only One Database related Properties in application.properties

Q14) How Can I interact with Two Databases in One Boot Application?

Ans:

You need two dataSources for Two Databases.

You can use One AutoConfigured DataSource and One Explit Configured

Q15) How Can I exclude the Jars which are downloaded by Spring Boot Starters?

Ans:

You can not change the Built-In Spring Boot Starters related Jars.

You Can write your own Spring Boot Starters.

Q16) Can I write my own Spring Boot Starters?

Ans:

Yes

Q17) What is the Default DataSource in Spring Boot 1.x?

Ans:

TomcatCP

Q18) What is the Default DataSource in Spring Boot 2x?

Ans:

HikariCP

Q19) Can I Change the Default DataSource to other Required DataSource?

Ans:

Yes, You Can

Q20) What to do if I want to use Tomcat dataSource in Boot 2.x?

Ans:

Q21) What to do if I want to use DBCP2 dataSource in Boot 2.x?

Ans:

Q22) What happens when I provide Following 3 DataSources?

HikariCP TomcatCP DBCP2

Ans:

HikariCP – Will be Selected

Q23) What happens when I provide Following 2 DataSources?

DBCP2 TomcatCP

Ans:

TomcatCP – Will be Selected

Q24) What happens when I am not providing Any DataSource after excluding Hikari?

Ans:

Error

Q25) What types of Parameters can be used when we write the Queries?

Ans:

Parameters are of Two Types:

- 1) Postional Parameters.
- 2) Named Parameters.

Q26) When Can I Use JdbcTemplate?

Ans:

When you want to use Postional Parameters in SQL

Q27) When Can I Use NamedParameterJdbcTemplate?

Ans:

When you want to use Named Parameters in SQL

Q28)Who manages the Version of MySQL when I am not Specifying the MySQL Version?

```
<dependency>  
  <groupId>mysql</groupId>  
  <artifactId>mysql-connector-java</artifactId>  
</dependency>
```

Ans:

Version is managed by Spring Boot based on the Spring boot Parent Version.

Defaults are as follows:

For Spring Boot 2.X => MySQL 8.x

For Spring Boot 1.5.x => MySQL 5.x

Q29) Can I use MySQL5 with Spring Boot? If so how it is?

Ans:

Yes, You can specify the MySQL Version required explicitly.

```
<dependency>  
  <groupId>mysql</groupId>  
  <artifactId>mysql-connector-java</artifactId>  
  <version>5.1.15</version>  
</dependency>
```



Q30) What are the Annotations replaced by @SpringBootApplication?

Ans:

Mark this Configuration class with @SpringBootApplication which replacement of the following 3 annotations.

- @Configuration
- @ComponentScan(basePackages = "com.jlcindia.springboot")
- @EnableAutoConfiguration

Q31) What is the CommandLineRunner interface?

Ans:

- 1) Implement CommandLineRunner
- 2) Override run() method

public void run(String... args) throws Exception {

3) write Code to perform some tasks at Boot Application Startup:

- Increase JVM Memory
- Load the cache data
- Check DB Connections
- Insert Some Data into DB
- Cross Check Third Party Apps Status
- etc

Q32) Who is responsible for Launching Spring Boot Application?

Ans:

SpringApplication.run(MyBootApplication.class, args);

Parameters of run() method

- 1) MyBootApplication => Boot Main class
- 2) Command Line Args which are coming to main() method.

Q33) What are the Stereotype Annotations?

Ans:

@Service
@Controller
@Repository
@Component

Q34) Can I develop spring boot app without CommandLineRunner ?

Ans:

Yes



Q35) What Happens Internally When you run the Boot Application?

Ans:

When you launch the Spring Boot Application then following Tasks will happen

A) main() method of Boot Main class will be called.

B) We must call static run() method on SpringApplication class by passing two parameters called Boot Main class and command line args

C) run() method of SpringApplication will do the following

1) Create the Spring Container (i.e ApplicationContext object)

```
ApplicationContext ctx= new AnnotationConfigApplicationContext();
```

At the time of Creating Application Context,i.e Spring Container Start-UP following things will happen.

- a) Scans all the Components for Sterotype Annotations . if found then prepares the List of Beans . **(HelloDAO, HelloService, JLCAppConfig,MyBootApplication) ***
- b) Loads all the beans found from Component scan.
 - i) HelloDAO bean will be loaded.
 - ii) HelloService bean will be loaded.
- c) Detects Bean Definitions from One or More Configuration class and Prepares the List of Bean Definitions.**(HaiDAO,HaiService>Hello,Hai) ****
- d) Loads all the beans from List of Bean Definitions
 - i) HaiDAO bean will be loaded.
 - ii) HaiService bean will be loaded.
 - iii) Hello bean will be loaded.
 - iv) Hai bean will be loaded.

2) Prepares the CONDITIONS EVALUATION REPORT with Positive Matches and Negative Matches.

Positive matches:

- a) DataSource
- b) JdbcTemplate
- c) NamedParameterJdbcTemplate
- d) DataSourceTransactionManager



3) AutoConfigure the Beans which are in Positive matches List.

- a) DataSource bean will be loaded.
- b) JdbcTemplate bean will be loaded.
- c) NamedParameterJdbcTemplate bean will be loaded.
- d) DataSourceTransactionManager bean will be loaded.

4) Checks whether Boot Main class is implementing CommandLineRunner or not.

If Boot Main class is implementing CommandLineRunner then only Overriden run() method will be called.